@azdocuments

PYTHON PROGRAMMING

SUBJECT CODE: 1BPLC105B/205B

MODULE-2

Name:

USN:

College:

AZ Documents

Your Engineering Study Partner

www.azdocuments.in

PYTHON PROGRAMMING

1BPLC105B/205B

MODULE 2

Syllabus:

Strings: Working with strings as single things, working with the parts of a string, Length, Traversal and the for loop, Slices, String comparison, Strings are immutable, the in and not in operators, A find function, Looping and counting, Optional parameters, The built-in find method, The split method, Cleaning up your strings, The string format method.
Tuples: Tuples are used for grouping data, Tuple assignment, Tuples as return values, Composability of Data
Structures.
Lists: List values, accessing elements, List length, List membership, List operations, List slices, Lists are mutable, List deletion, Objects and references, Aliasing, cloning lists, Lists and for loops, List parameters, List methods, Pure functions and modifiers, Functions that produce lists, Strings and lists, list and range, Nested
lists, Matrices.
Chapter: 5.1, 5.2, 5.3

2.1 Strings

2.1.1 A compound data type

- So far, we have seen built-in data types like:
    - int → integers
    - float → decimal numbers
    - bool → Boolean (True/False)
    - str → strings
- Strings, lists, and pairs are different from other types because:
    - They are made up of smaller pieces.
    - For example, a string is made up of smaller strings, each containing one character.
    - These types are called compound data types because they are made of multiple smaller components.
- Depending on the situation, we can:
    - Treat a compound data type as a single object, or
    - Access and work with its individual parts.
    - This flexibility (being able to use both ways) makes compound data types very useful.

2.1.2 Working with Strings as Single Things

We previously saw that each turtle instance has its own attributes and a number of methods that can be applied to the instance.
For example, we could set the turtle's color, and we wrote:

tess.turn(90)

Just like a turtle, a string is also an object.
So, each string instance has its own attributes and methods.

```
>>> our_string = "Hello, World!"
>>> all_caps = our_string.upper()
>>> all_caps
'HELLO, WORLD!'
```

String Methods

The upper() method can be invoked on any string object to create a new string in which all the characters are in uppercase.
(The original string our_string remains unchanged.)

There are also other methods such as:

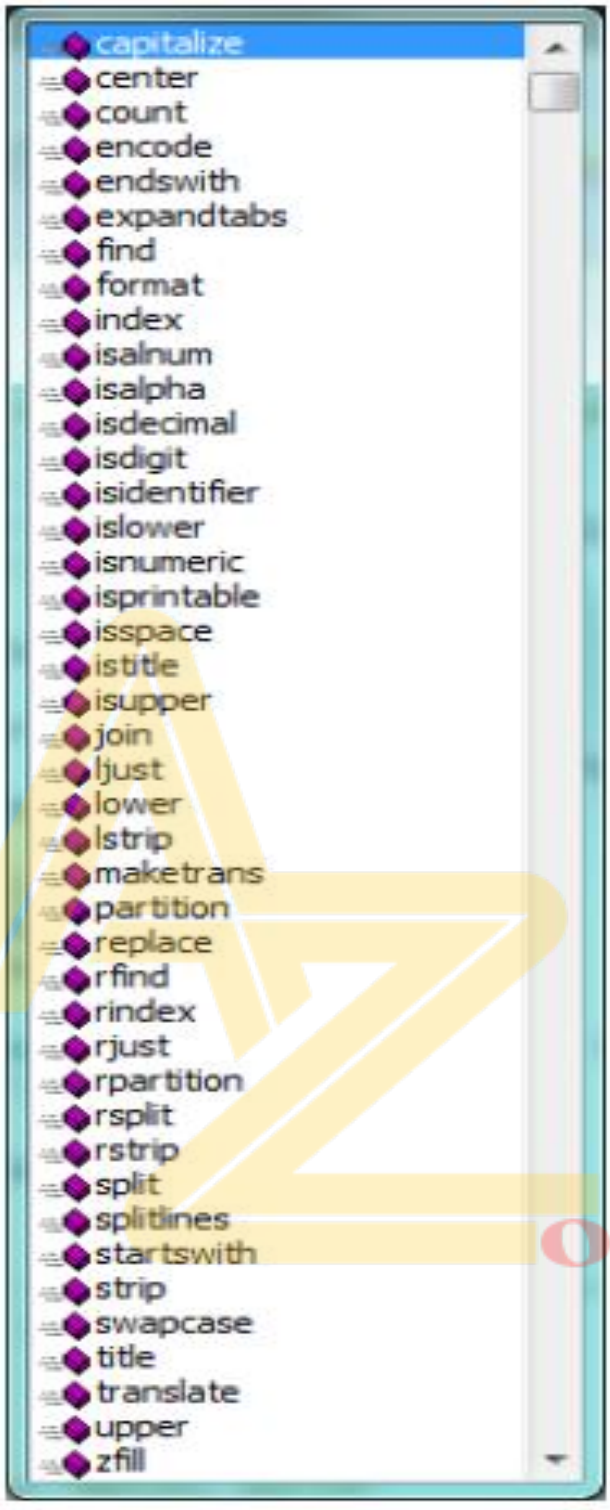
- lower() – converts all characters to lowercase.
- capitalize() – capitalizes the first character of the string.
- swapcase() – converts uppercase letters to lowercase and vice versa.

To learn what methods are available, you can:

- Consult the Help documentation and look for string methods, or
- Simply type the following into an editor (like Spyder or PyScripter):

```
1 our_string = "Hello, World!"
2 new_string = our_string.
```

When you type the period (.) after the variable name, your editor might pop up a selection window (often by pressing Tab) showing all available methods — there are around 70 of them (but we'll only use a few important ones!).



When you type the name of a method, some additional information about its parameters, return type, and its docstring may be displayed by your scripting environment.

For example, in a Jupyter Notebook, you can view this information by pressing Shift + Tab after typing a function or method name.

```
greet = "Hello, World"
xx= greet.swapcase()
print(xx)
```

```
** No/Unknown parameters **
S.swapcase() -> str

Return a copy of S with uppercase characters converted to lowercase
and vice versa.
```

2.1.3 Working with the parts of a string

The indexing operator (Python uses square brackets to enclose the index) selects a single character substring from a string:

```
>>> fruit = "banana"
>>> letter = fruit[1]
>>> print(letter)
```

The expression fruit[1] selects character number 1 from fruit, and creates a new string containing just this one character. The variable letter refers to the result. When we display letter, we could get a surprise:

```
a
```

Computer scientists always start counting from zero! The letter at subscript position zero of "banana" is b. So at position [1] we have the letter a. If we want to access the zero-eth letter of a string, we just place 0, or any expression that evaluates to 0, inbetween the brackets:

```
>>> letter = fruit[0]
>>> print(letter)
b
```

The expression in brackets is called an index. An index specifies a member of an ordered collection, in this case the collection of characters in the string. The index indicates which one you want, hence the name. It can be any integer expression. We can use enumerate to visualize the indices:

```
>>> fruit = "banana"
>>> list(enumerate(fruit))
[(0, 'b'), (1, 'a'), (2, 'n'), (3, 'a'), (4, 'n'), (5, 'a')]
```

Do not worry about enumerate at this point, we will see more of it in the chapter on lists. Note that indexing returns a string Python has no special type for a single character. It is just a string of length 1. We've also seen lists previously. The same indexing notation works to extract elements from a list:

```
>>> prime_numbers = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31]
>>> prime_numbers[4]
11
>>> friends = ["Joe", "Zoe", "Brad", "Angelina", "Zuki", "Thandi", "Paris"]
>>> friends[3]
'Angelina'
```

2.1.4 Length

The len function, when applied to a string, returns the number of characters in a string:

```
>>> word = "banana"
>>> len(word)
6
```

To get the last letter of a string, you might be tempted to try something like this:

```
1 size = len(word)
2 last = word[size]       # ERROR!
```

That won't work. It causes the runtime error IndexError: string index out of range. The reason is that there is no character at index position 6 in "banana". Because we start counting at zero, the six indexes are numbered 0 to 5. To get the last character, we have to subtract 1 from the length of word:

```
1 size = len(word)
2 last = word[size-1]
```

Alternatively, we can use negative indices, which count backward from the end of the string. The expression word[-1] yields the last letter, word[-2] yields the second to last, and so on. As you might have guessed, indexing with a negative index also works like this for lists.

2.1.5 Traversal and the for loop

A lot of computations involve processing a string one character at a time. Often they start at the beginning, select each character in turn, do something to it, and continue until the end. This pattern of processing is called a traversal. One way (a very bad way) to encode a traversal is with a while statement:

```
ix = 0
while ix < len(fruit):
    letter = fruit[ix]
    print(letter)
    ix += 1
```

This loop traverses the string and displays each letter on a new line.

It uses ix as the index variable, though this does not make it any clearer.

The loop condition is:

ix < len(fruit)

So, when ix becomes equal to the length of the string, the condition becomes false, and the loop stops executing.

The last character accessed is the one with the index len(fruit) - 1, which represents the last character in the string.

However, this code is:

Longer than necessary, and Not very clear.

We've already seen that a for loop can easily iterate over the elements in a list, and it can do the same for strings as well.

But we've previously seen how the for loop can easily iterate over the elements in a list and it can do so for strings as well:

```
word="Banana"
for letter in word:
    print(letter)
```

Each time through the loop, the next character in the string is assigned to the variable c. The loop continues until no characters are left. Here we can see the expressive power the for loop gives us compared to the while loop when traversing a string.

The following example shows how to use concatenation and a for loop to generate an abecedarian series. Abecedarian refers to a series or list in which the elements appear in alphabetical order. For example, in Robert McCloskey's book Make Way for Ducklings, the names of the ducklings are Jack, Kack, Lack, Mack, Nack, Ouack, Pack, and Quack. This loop outputs these names in order:

```
prefixes = "JKLMNOPQ"
suffix = "ack"

for p in prefixes:
    print(p + suffix)
```

The output of this program is:

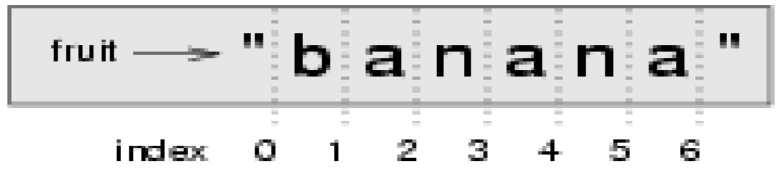
```
Jack
Kack
Lack
Mack
Nack
Oack
Pack
Qack
```

2.1.6 Slices

A substring of a string is obtained by taking a slice. Similarly, we can slice a list to refer to some sublist of the items in the list:

```
>>> phrase = "Pirates of the Caribbean"
>>> print(phrase[0:7])
Pirates
>>> print(phrase[11:14])
the
>>> print(phrase[13:24])
e Caribbean
>>> friends = ["Joe", "Zoe", "Brad", "Angelina", "Zuki", "Thandi", "Paris"]
>>> print(friends[2:4])
['Brad', 'Angelina']
```

The operator [n:m] returns the part of the string from the n'th character to the m'th character, including the first but excluding the last. This behavior makes sense if you imagine the indices pointing between the characters, as in the following diagram:



If you imagine a string as a piece of paper, the slice operator [n:m] copies out the part of the paper between the n and m positions.

Provided both n and m are within the bounds of the string, the result will have a length of (m - n).

Slicing Tricks

1. Omitting the first index ([:m])
   → The slice starts from the beginning of the string (or list).
2. Omitting the second index ([n:])
   → The slice extends to the end of the string (or list).
3. If n is greater than the length of the string (or list):
   → The slice will take all values up to the end,
   and will not raise an "out of range" error (unlike normal indexing).

```
>>> word = "banana"
>>> word[:3]
'ban'
>>> word[3:]
'ana'
>>> word[3:999]
'ana'
```

2.1.7 String comparison

The comparison operators work on strings. To see if two strings are equal:

```
1 if word == "banana":
2     print("Yes, we have no bananas!")
```

Other comparison operations are useful for putting words in lexicographical order:

```
1 if word < "banana":
2     print("Your word, " + word + ", comes before banana.")
3 elif word > "banana":
4     print("Your word, " + word + ", comes after banana.")
5 else:
6     print("Yes, we have no bananas!")
```

This is similar to the alphabetical order you would use with a dictionary, except that all the uppercase letters come before all the lowercase letters. As a result:

Your word, Zebra, comes before banana.

A common way to address this problem is to convert strings to a standard format, such as all lowercase, before performing the comparison. A more difficult problem is making the program realize that zebras are not fruit.

2.1.8 Strings are immutable

It is tempting to use the [] operator on the left side of an assignment, with the intention of changing a character in a string. For example:

```
greeting = "Hello, world!"
greeting[0] = 'J'           # ERROR!
print(greeting)
```

Instead of producing the output Jello, world!, this code produces the runtime error TypeError: 'str' object does not support item assignment. Strings are immutable, which means you can't change an existing string. The best you can do is create a new string that is a variation on the original:

```
greeting = "Hello, world!"
new_greeting = "J" + greeting[1:]
print(new_greeting)
```

The solution here is to concatenate a new first letter onto a slice of greeting. This operation has no effect on the original string.

2.1.9 The in and not in operators

The in operator tests for membership. When both of the arguments to in are strings, in checks whether the left argument is a substring of the right argument.

```
>>> "p" in "apple"
True
>>> "i" in "apple"
False
```

(continues on next page)

(continued from previous page)

```
>>> "ap" in "apple"
True
>>> "pa" in "apple"
False
```

Note that a string is a substring of itself, and the empty string is a substring of any other string. (Also note that computer scientists like to think about these edge cases quite carefully!)

```
>>> "a" in "a"
True
>>> "apple" in "apple"
True
>>> "" in "a"
True
>>> "" in "apple"
True
```

The not in operator returns the logical opposite results of in:

```
>>> "x" not in "apple"
True
```

Combining the in operator with string concatenation using +, we can write a function that removes all the vowels from a string:

```
1 def remove_vowels(phrase):
2     vowels = "aeiou"
3     string_sans_vowels = ""
4     for letter in phrase:
5         if letter.lower() not in vowels:
6             string_sans_vowels += letter
7     return string_sans_vowels
```

Important to note is the letter.lower() in line 5, without it, any uppercase vowels would not be removed.

2.1.10 A find function

```
def my_find(haystack, needle):
    """
      Find and return the index of needle in haystack.
      Return -1 if needle does not occur in haystack.
    """
    for index, letter in enumerate(haystack):
        if letter == needle:
            return index
    return -1
```

```
haystack = "Bananarama!"
print(haystack.find('a'))
print(my_find(haystack,'a'))
```

Understanding the find() Function

In a sense, the find function works opposite to the indexing operator.

- The indexing operator takes an index and extracts the corresponding character.
- The find function does the reverse — it takes a character and returns the index where that character appears in the string.

If the character is not found, the function returns -1.

Return Statement Inside a Loop

This function also helps us understand how a return statement works inside a loop.
If the condition letter == needle is true, then the function:

- Returns immediately,
- Breaks out of the loop before checking the rest of the string.

If the character does not appear in the string, the loop finishes normally, and then the function returns -1.

Eureka Traversal / Short-Circuit Evaluation

This pattern of computation is called Eureka Traversal or Short-Circuit Evaluation,
because as soon as we find what we are looking for, we can say “Eureka!”,
take the short-circuit, and stop searching further saving both time and effort.

2.1.11 Looping and counting

The following program counts the number of times the letter a appears in a string, and is another example of the counter pattern introduced in Counting digits:

```
def count_a(text):
    count = 0
    for letter in text:
        if letter == "a":
            count += 1
    return(count)

print(count_a("banana") == 3)
```

2.1.12 Optional parameters

To find the locations of the second or third occurrence of a character in a string, we can modify the find function, adding a third parameter for the starting position in the search string:

```
def find2(haystack, needle, start):
    for index,letter in enumerate(haystack[start:]):
        if letter == needle:
            return index + start
    return -1



print(find2("banana", "a", 2) == 3)
```

The call find2("banana", "a", 2) now returns 3, the index of the first occurrence of “a” in “banana” starting the search at index 2. What does find2("banana", "n", 3) return? If you said, 4, there is a good chance you understand how find2 works.

Better still, we can combine find and find2 using an optional parameter:

```
def find(haystack, needle, start=0):
    for index,letter in enumerate(haystack[start:]):
        if letter == needle:
            return index + start
    return -1
```

When a function has an optional parameter, the caller may provide a matching argument. If the third argument is provided to find, it gets assigned to start. But if the caller leaves the argument out, then start is given a default value indicated by the assignment start=0 in the function definition.

So the call find("banana", "a", 2) to this version of find behaves just like find2, while in the call find("banana", "a"), start will be set to the default value of 0.

Adding another optional parameter to find makes it search from a starting position, up to but not including the end position:

```
def find(haystack, needle, start=0, end=-1):
    for index,letter in enumerate(haystack[start:end])
        if letter == needle:
            return index + start
    return -1
```

T

The semantics of start and end in this function are precisely the same as they are in the range function.

2.1.13 The built-in find method

Built-in find() Method in Strings

Now that we've written our own find function, we can note that Python strings already have a built-in find() method.

This built-in method can do everything our version does — and even more!

You can try all the previous examples again and check the results.

Advantages of the Built-in find() Method

The built-in find() method is more general than our version because:

- It can find substrings, not just single characters.
- It also allows you to specify a starting position for the search.

```
>>> "banana".find("nan")
2
>>> "banana".find("na", 3)
4
```

2.1.14 The split method

One of the most useful methods on strings is the split method: it splits a single multi-word string into a list of individual words, removing all the whitespace between them. (Whitespace means any tabs, newlines, or spaces.) This allows us to read input as a single string, and split it into words.

```
>>> phrase = "Well I never did said Alice"
>>> words = phrase.split()
>>> words
['Well', 'I', 'never', 'did', 'said', 'Alice']
```

2.1.15 Cleaning up your strings

Removing Punctuation from Strings

When working with strings, we often encounter punctuation marks, tab characters, or newline characters
especially when reading text from files or the Internet (as we'll see in future chapters).

However, if we are writing a program to:

- Count word frequencies, or
- Check the spelling of each word,

we need to remove these unwanted characters first.

Since strings are immutable (cannot be changed after creation),
we cannot modify the original string that contains punctuation.

Instead, we must:

1. Traverse the original string, and
2. Create a new string that omits any punctuation characters.

```
punctuation = "!\"#$%&'()*+,-./:;<=>?@[\\]^_`{|}~"

def remove_punctuation(phrase):
    phrase_sans_punct = ""
    for letter in phrase:
        if letter not in punctuation:
            phrase_sans_punct += letter
    return phrase_sans_punct
```

Setting up that first assignment is messy and error-prone. Fortunately, the Python string module already does it for us. So we will make a slight improvement to this program — we'll import the string module and use its definition:

```
import string

def remove_punctuation(phrase):
    phrase_sans_punct = ""
    for letter in phrase:
        if letter not in string.punctuation:
            phrase_sans_punct += letter
    return phrase_sans_punct
```

Try the examples below: “Well, I never did!”, said Alice. “Are you very, very, sure?” Composing together this function and the split method from the previous section makes a useful combination — we’ll clean out the punctuation, and split will clean out the newlines and tabs while turning the string into a list of words:

```
my_story = """
Pythons are constrictors, which means that they will 'squeeze' the life
out of their prey. They coil themselves around their prey and with
each breath the creature takes the snake will squeeze a little tighter
until they stop breathing completely. Once the heart stops the prey
is swallowed whole. The entire animal is digested in the snake's
stomach except for fur or feathers. What do you think happens to the fur,
feathers, beaks, and eggshells? The 'extra stuff' gets passed out as ---
you guessed it --- snake POOP! """

words = remove_punctuation(my_story).split()
print(words)
```

The output:

```
['Pythons', 'are', 'constrictors', ... , 'it', 'snake', 'POOP']
```

2.1.16 The string format method

The easiest and most powerful way to format a string in Python 3 is to use the format method. To see how this works, let’s start with a few examples:

```
phrase = "His name is {0}!".format("Arthur")
print(phrase)

name = "Alice"
```

(continues on next page)

(continued from previous page)

```
5  age = 10
6  phrase = "I am {1} and I am {0} years old.".format(age, name)
7  print(phrase)
8  phrase = "I am {0} and I am {1} years old.".format(age, name)
9  print(phrase)
10
11 x = 4
12 y = 5
13 phrase = "2**10 = {0} and {1} * {2} = {3:f}".format(2**10, x, y, x * y)
14 print(phrase)
```

Running the script produces:

```
His name is Arthur!
I am Alice and I am 10 years old.
I am 10 and I am Alice years old.
2**10 = 1024 and 4 * 5 = 20.000000
```

The template string contains place holders, ... {0} ... {1} ... {2} ... etc. The format method substitutes its arguments into the place holders. The numbers in the place holders are indexes that determine which argument gets substituted — make sure you understand line 6 above!

But there's more! Each of the replacement fields can also contain a format specification — it is always introduced by the : symbol (Line 13 above uses one.) This modifies how the substitutions are made into the template, and can control things like:

• whether the field is aligned to the left

• the width allocated to the field within the result string (a number like 10)

• the type of conversion (we'll initially only force conversion to float, f, as we did in line 13 of the code above, or perhaps we'll ask integer numbers to be converted to hexadecimal using x)

• if the type conversion is a float, you can also specify how many decimal places are wanted (typically, .2f is useful for working with currencies to two decimal places.)

Let's do a few simple and common examples that should be enough for most needs. If you need to do anything more esoteric, use *help* and read all the powerful, gory details.

```
name1 = "Paris"
name2 = "Whitney"
name3 = "Hilton"

print("Pi to three decimal places is {0:.3f}".format(3.1415926))
print("123456789 123456789 123456789 123456789 123456789 123456789")
print("|||{0:<15}|||{1:^15}|||{2:>15}|||Born in {3}|||"
        .format(name1,name2,name3,1981))
print("The decimal value {0} converts to hex value {0:x}"
        .format(123456))
```

This script produces the output:

```
Pi to three decimal places is 3.142
123456789 123456789 123456789 123456789 123456789 123456789
|||Paris          |||    Whitney    |||         Hilton|||Born in 1981|||
The decimal value 123456 converts to hex value 1e240
```

```
letter = """
Dear {0} {2}.
 {0}, I have an interesting money-making proposition for you!
 If you deposit $10 million into my bank account, I can
 double your money ...
"""

print(letter.format("Paris", "Whitney", "Hilton"))
print(letter.format("Bill", "Henry", "Gates"))
```

This produces the following:

```
Dear Paris Hilton.
 Paris, I have an interesting money-making proposition for you!
 If you deposit $10 million into my bank account, I can
 double your money ...


Dear Bill Gates.
 Bill, I have an interesting money-making proposition for you!
 If you deposit $10 million into my bank account I can
 double your money ...
```

As you might expect, you'll get an index error if your placeholders refer to arguments that you do not provide:

```
>>> "hello {3}".format("Dave")
Traceback (most recent call last):
  File "<interactive input>", line 1, in <module>
IndexError: tuple index out of range
```

The following example illustrates the real utility of string formatting. First, we'll try to print a table without using string formatting:

```
print("i\ti**2\ti**3\ti**5\ti**10\ti**20")
for i in range(1, 11):
    print(i, "\t", i**2, "\t", i**3, "\t", i**5, "\t",
                                          i**10, "\t", i**20)
```

This program prints out a table of various powers of the numbers from 1 to 10. (This assumes that the tab width is 8. You might see something even worse than this if you tab width is set to 4.) In its current form it relies on the tab character (\t) to align the columns of values, but this breaks down when the values in the table get larger than the tab width:

```
i       i**2    i**3    i**5    i**10   i**20
1       1       1       1       1       1
2       4       8       32      1024    1048576
3       9       27      243     59049   3486784401
4       16      64      1024    1048576         1099511627776
5       25      125     3125    9765625         95367431640625
6       36      216     7776    60466176        3656158440062976
7       49      343     16807   282475249       79792266297612001
8       64      512     32768   1073741824      1152921504606846976
9       81      729     59049   3486784401      12157665459056928801
10      100     1000    100000  10000000000     100000000000000000000
```

One possible solution would be to change the tab width, but the first column already has more space than it needs. The best solution would be to set the width of each column independently. As you may have guessed by now, string formatting provides a much nicer solution. We can also right-justify each field:

```
layout = "{0:>4}{1:>6}{2:>6}{3:>8}{4:>13}{5:>24}"

print(layout.format("i", "i**2", "i**3", "i**5", "i**10", "i**20"))
for i in range(1, 11):
    print(layout.format(i, i**2, i**3, i**5, i**10, i**20))
```

Running this version produces the following (much more satisfying) output:

```
   i  i**2  i**3    i**5        i**10                  i**20
   1     1     1       1           1                      1
   2     4     8      32        1024                1048576
   3     9    27     243       59049             3486784401
   4    16    64    1024     1048576          1099511627776
   5    25   125    3125     9765625         95367431640625
   6    36   216    7776    60466176       3656158440062976
   7    49   343   16807   282475249     79792266297612001
   8    64   512   32768  1073741824   1152921504606846976
   9    81   729   59049  3486784401  12157665459056928801
  10   100  1000  100000 10000000000 100000000000000000000
```

2.1.17 Glossary

compound data type A data type in which the values are made up of components, or elements, that are themselves values.

default value The value given to an optional parameter if no argument for it is provided in the function call.

docstring A string constant on the first line of a function or module definition (and as we will see later, in class and method definitions as well). Docstrings provide a convenient way to associate documentation with code. Docstrings are also used by programming tools to provide interactive help.

dot notation Use of the dot operator, ., to access methods and attributes of an object.

immutable data value A data value which cannot be modified. Assignments to elements or slices (sub-parts) of immutable values cause a runtime error.

index A variable or value used to select a member of an ordered collection, such as a character from a string, or an element from a list.

mutable data value A data value which can be modified. The types of all mutable values are compound types. Lists and dictionaries are mutable; strings and tuples are not.

optional parameter A parameter written in a function header with an assignment to a default value which it will receive if no corresponding argument is given for it in the function call.

short-circuit evaluation A style of programming that shortcuts extra work as soon as the outcome is know with certainty. In this chapter our find function returned as soon as it found what it was looking for; it didn't traverse all the rest of the items in the string.

slice A part of a string (substring) specified by a range of indices. More generally, a subsequence of any sequence type in Python can be created using the slice operator (sequence[start:stop]).

traverse To iterate through the elements of a collection, performing a similar operation on each.

whitespace Any of the characters that move the cursor without printing visible characters. The constant string. whitespace contains all the white-space characters.

2.2 Tuples

2.2.1 Tuples are used for Grouping Data

- Definition:
  A tuple is a data structure used to group multiple values together into a single compound value.
- Syntax:
  A tuple is written as a comma-separated sequence of values, usually enclosed in parentheses ( ).
  Example:

```
>>> year_born = ("Paris Hilton", 1981)
```

This is an example of a data structure a mechanism for grouping and organizing data to make it easier to use.

The pair is an example of a tuple. Generalizing this, a tuple can be used to group any number of items into a single compound value. Syntactically, a tuple is a comma-separated sequence of values. Although it is not necessary, it is conventional to enclose tuples in parentheses:

```
>>> julia = ("Julia", "Roberts", 1967, "Duplicity", 2009, "Actress",
↪"Atlanta, Georgia")
```

A tuple is a data structure used in Python to group related data together into a single unit. It is written as a comma-separated sequence of values, usually enclosed in parentheses. The parentheses are used to avoid confusion, especially when a tuple is nested within another tuple. Without them, it would be difficult to identify where the inner tuple begins and ends. An empty tuple can be created using empty parentheses like this:

empty_tuple = ()

Tuples are useful for representing information that belongs together, such as a student record or personal details. In other programming languages, similar structures are often called records or structs. Although tuples group related data, they do not have field names, so we understand their meaning based on context.

Just like strings, tuples support sequence operations such as indexing and slicing. For example, if we have a tuple:

```
>>> julia = ("Julia", "Roberts", 1967, "Duplicity", 2009, "Actress",
↪"Atlanta, Georgia")
```

we can access an element using its index:

```
>>> julia[2]
1967
```

However, tuples are immutable, which means that once a tuple is created, its elements cannot be modified. If we try to assign a new value to an element, Python gives an error:

```
>>> julia[0] = "X"
TypeError: 'tuple' object does not support item assignment
```

Although tuples cannot be changed directly, we can create a new tuple using parts of the old one. For example, if Julia acts in a new film, we can update her details by slicing the old tuple and combining it with new information:

```
>>> julia = julia[:3] + ("Eat Pray Love", 2010) + julia[5:]
>>> julia
("Julia", "Roberts", 1967, "Eat Pray Love", 2010, "Actress", "Atlanta,␣
↪Georgia")
```

Now, the tuple becomes:

To create a tuple with a single element (but you're probably not likely to do that too often), we have to include the final comma, because without the final comma, Python treats the (5) below as an integer in parentheses:

```
>>> tup = (5,)
>>> type(tup)
<class 'tuple'>
>>> x = (5)
>>> type(x)
<class 'int'>
```

2.2.2 Tuple assignment

Python has a very powerful tuple assignment feature that allows a tuple of variables on the left of an assignment to be assigned values from a tuple on the right of the assignment. (We already saw this used for pairs, but it generalizes.)

```
(name, surname, year_born, movie, year_movie, profession, birthplace) = julia
```

This does the equivalent of seven assignment statements, all on one easy line. One requirement is that the number of variables on the left must match the number of elements in the tuple.

One way to think of tuple assignment is as tuple packing/unpacking.

In tuple packing, the values on the left are 'packed' together in a tuple:

```
>>> bob = ("Bob", 19, "CS")    # tuple packing
```

In tuple unpacking, the values in a tuple on the right are 'unpacked' into the variables/names on the right:

```
>>> bob = ("Bob", 19, "CS")
>>> (name, age, studies) = bob    # tuple unpacking
>>> name
```

```
'Bob'
>>> age
19
>>> studies
'CS'
```

Once in a while, it is useful to swap the values of two variables. With conventional assignment statements, we have to use a temporary variable. For example, to swap a and b:

```
temp = a
a = b
b = temp
```

Tuple assignment solves this problem neatly:

```
(a, b) = (b, a)
```

The left side is a tuple of variables; the right side is a tuple of values. Each value is assigned to its respective variable.

All the expressions on the right side are evaluated before any of the assignments. This feature makes tuple assignment quite versatile.

Naturally, the number of variables on the left and the number of values on the right have to be the same:

```
>>> (one, two, three, four) = (1, 2, 3)
ValueError: need more than 3 values to unpack
```

2.2.3 Tuples as return values

Functions can always only return a single value, but by making that value a tuple, we can effectively group together as many values as we like, and return them together. This is very useful — we often want to know some batsman's highest and lowest score, or we want to find the mean and the standard deviation, or we want to know the year, the month, and the day, or if we're doing some some ecological modelling we may want to know the number of rabbits and the number of wolves on an island at a given time.

For example, we could write a function that returns both the area and the circumference of a circle of radius r:

```
def circle_stats(r):
    """ Return (circumference, area) of a circle of radius r """
    circumference = 2 * math.pi * r
    area = math.pi * r * r
    return (circumference, area)
```

2.2.4 Composability of Data Structures

We saw in an earlier chapter that we could make a list of pairs, and we had an example where one of the items in the tuple was itself a list:

```
students = [
    ("John", ["CompSci", "Physics"]),
    ("Vusi", ["Maths", "CompSci", "Stats"]),
    ("Jess", ["CompSci", "Accounting", "Economics", "Management"]),
    ("Sarah", ["InfSys", "Accounting", "Economics", "CommLaw"]),
    ("Zuki", ["Sociology", "Economics", "Law", "Stats", "Music"])]
```

Tuples items can themselves be other tuples. For example, we could improve the information about our movie stars to hold the full date of birth rather than just the year, and we could have a list of some of her movies and dates that they were made, and so on:

```
julia_more_info = ( ("Julia", "Roberts"), (8, "October", 1967),
                    "Actress", ("Atlanta", "Georgia"),
                    [ ("Duplicity", 2009),
                      ("Notting Hill", 1999),
                      ("Pretty Woman", 1990),
                      ("Erin Brockovich", 2000),
                      ("Eat Pray Love", 2010),
                      ("Mona Lisa Smile", 2003),
                      ("Oceans Twelve", 2004) ])
```

Notice in this case that the tuple has just five elements but each of those in turn can be another tuple, a list, a string, or any other kind of Python value. This property is known as being heterogeneous, meaning that it can be composed of elements of different types.

2.2.5 Glossary

data structure An organization of data for the purpose of making it easier to use.

immutable data value A data value which cannot be modified. Assignments to elements or slices (sub-parts) of immutable values cause a runtime error.

mutable data value A data value which can be modified. The types of all mutable values are compound types. Lists and dictionaries are mutable; strings and tuples are not.

tuple An immutable data value that contains related elements. Tuples are used to group together related data, such as a person's name, their age, and their gender.

tuple assignment An assignment to all of the elements in a tuple using a single assignment statement. Tuple assign-ment occurs simultaneously rather than in sequence, making it useful for swapping values.

2.3 Lists

A list is an ordered collection of values. The values that make up a list are called its elements, or its items. We will use the term element or item to mean the same thing. Lists are similar to strings, which are ordered collections of characters, except that the elements of a list can be of any type. Lists and strings — and other collections that maintain the order of their items — are called sequences.

2.3.1 List values

There are several ways to create a new list; the simplest is to enclose the elements in square brackets ([and]):

```
1 numbers = [10, 20, 30, 40]
2 words = ["spam", "bungee", "swallow"]
```

The first example is a list of four integers. The second is a list of three strings. The elements of a list don't have to be the same type. The following list contains a string, a float, an integer, and (amazingly) another list:

```
1 stuffs = ["hello", 2.0, 5, [10, 20]]
```

A list within another list is said to be nested.

Finally, a list with no elements is called an empty list, and is denoted [].

We have already seen that we can assign list values to variables or pass lists as parameters to functions:

```
>>> vocabulary = ["apple", "cheese", "dog"]
>>> numbers = [17, 123]
>>> an_empty_list = []
>>> print(vocabulary, numbers, an_empty_list)
["apple", "cheese", "dog"] [17, 123] []
```

2.3.2 Accessing elements

The syntax for accessing the elements of a list is the same as the syntax for accessing the characters of a string the index operator: [] (not to be confused with an empty list). The expression inside the brackets specifies the index. Remember that the indices start at 0:

```
>>> numbers[0]
17
```

Any expression evaluating to an integer can be used as an index:

```
>>> numbers[9-8]
123
>>> numbers[1.0]
Traceback (most recent call last):
  File "<interactive input>", line 1, in <module>
TypeError: list indices must be integers, not float
```

If you try to access or assign to an element that does not exist, you get a runtime error:

```
>>> numbers[2]
Traceback (most recent call last):
  File "<interactive input>", line 1, in <module>
IndexError: list index out of range
```

It is common (but wrong!) to use a loop variable as a list index.

```
horsemen = ["war", "famine", "pestilence", "death"]

for i in [0, 1, 2, 3]:
    print(horsemen[i])
```

Each time through the loop, the variable i is used as an index into the list, printing the i'th element. This pattern of computation is called a list traversal.

The above sample doesn't need or use the index i for anything besides getting the items from the list, so this more direct version — where the for loop gets the items — is much more clear!

```
horsemen = ["war", "famine", "pestilence", "death"]

for h in horsemen:
    print(h)
```

2.3.3 List length

The len() function in Python is used to find the length of a list. list, which is equal to the number of its elements. If you are going to use an integer index to access the list, it is a good idea to use this value as the upper bound of a loop instead of a constant. That way, if the size of the list changes, you won't have to go through the program changing all the loops; they will work correctly for any size list:

```
horsemen = ["war", "famine", "pestilence", "death"]

for i in range(len(horsemen)):
    print(horsemen[i])
```

The last time the body of the loop is executed, i is len(horsemen) - 1, which is the index of the last element.

```
horsemen = ["war", "famine", "pestilence", "death"]

for horseman in horsemen:
    print horseman
```

Although a list can contain another list, the nested list still counts as a single element in its parent list. The length of this list is 4:

```
>>> len(["car makers", 1, ["Ford", "Toyota", "BMW"], [1, 2, 3]])
4
```

2.3.4 List membership

in and not in are Boolean operators that test membership in a sequence. We used them previously with strings, but they also work with lists and other sequences:

```
>>> horsemen = ["war", "famine", "pestilence", "death"]
>>> "pestilence" in horsemen
True
>>> "debauchery" in horsemen
False
>>> "debauchery" not in horsemen
True
```

Using this produces a more elegant version of the nested loop program we previously used to count the number of students doing Computer Science in the section Nested Loops for Nested Data:

```
1  students = [
2      ("John", ["CompSci", "Physics"]),
3      ("Vusi", ["Maths", "CompSci", "Stats"]),
4      ("Jess", ["CompSci", "Accounting", "Economics", "Management"]),
5      ("Sarah", ["InfSys", "Accounting", "Economics", "CommLaw"]),
6      ("Zuki", ["Sociology", "Economics", "Law", "Stats", "Music"])]
7
8  # Count how many students are taking CompSci
9  counter = 0
10 for name, subjects in students:
11     if "CompSci" in subjects:
12             counter += 1
13
14 print("The number of students taking CompSci is", counter)
```

2.3.5 List operations

The + operator concatenates lists:

```
>>> first_list = [1, 2, 3]
>>> second_list = [4, 5, 6]
>>> both_lists = first_list + second_list
>>> both_lists
[1, 2, 3, 4, 5, 6]
```

Similarly, the * operator repeats a list a given number of times:

```
>>> [0] * 4
[0, 0, 0, 0]
>>> [1, 2, 3] * 3
[1, 2, 3, 1, 2, 3, 1, 2, 3]
```

The first example repeats [0] four times. The second example repeats the list [1, 2, 3] three times.

2.3.6 List slices

The slice operations we saw previously with strings let us work with sublists:

```
>>> a_list = ["a", "b", "c", "d", "e", "f"]
>>> a_list[1:3]
['b', 'c']
>>> a_list[:4]
['a', 'b', 'c', 'd']
>>> a_list[3:]
['d', 'e', 'f']
>>> a_list[:]
['a', 'b', 'c', 'd', 'e', 'f']
```

2.3.7 Lists are mutable

Unlike strings, lists are mutable, which means we can change their elements. Using the index operator on the left side of an assignment, we can update one of the elements:

```
>>> fruit = ["banana", "apple", "quince"]
>>> fruit[0] = "pear"
>>> fruit[2] = "orange"
>>> fruit
['pear', 'apple', 'orange']
```

The bracket operator applied to a list can appear anywhere in an expression. When it appears on the left side of an assignment, it changes one of the elements in the list, so the first element of fruit has been changed from "banana" to "pear", and the last from "quince" to "orange". An assignment to an element of a list is called item assignment. Item assignment does not work for strings:

```
>>> my_string = "TEST"
>>> my_string[2] = "X"
Traceback (most recent call last):
  File "<interactive input>", line 1, in <module>
TypeError: 'str' object does not support item assignment
```

but it does for lists:

```
>>> my_list = ["T", "E", "S", "T"]
>>> my_list[2] = "X"
>>> my_list
['T', 'E', 'X', 'T']
```

With the slice operator we can update a whole sublist at once:

```
>>> a_list = ["a", "b", "c", "d", "e", "f"]
>>> a_list[1:3] = ["x", "y"]
>>> a_list
['a', 'x', 'y', 'd', 'e', 'f']
```

We can also remove elements from a list by assigning an empty list to them:

```
>>> a_list = ["a", "b", "c", "d", "e", "f"]
>>> a_list[1:3] = []
>>> a_list
['a', 'd', 'e', 'f']
```

And we can add elements to a list by squeezing them into an empty slice at the desired location:

```
>>> a_list = ["a", "d", "f"]
>>> a_list[1:1] = ["b", "c"]
>>> a_list
['a', 'b', 'c', 'd', 'f']
>>> a_list[4:4] = ["e"]
>>> a_list
['a', 'b', 'c', 'd', 'e', 'f']
```

2.3.8 List deletion

Using slices to delete list elements can be error-prone. Python provides an alternative that is more readable. The del statement removes an element from a list:

```
>>> a = ["one", "two", "three"]
>>> del a[1]
>>> a
['one', 'three']
```

As you might expect, del causes a runtime error if the index is out of range.

You can also use del with a slice to delete a sublist:

```
>>> a_list = ["a", "b", "c", "d", "e", "f"]
>>> del a_list[1:5]
>>> a_list
['a', 'f']
```

As usual, the sublist selected by slice contains all the elements up to, but not including, the second index.

2.3.9 Objects and references

After we execute these assignment statements

```
1 a = "banana"
2 b = "banana"
```

we know that a and b will refer to a string object with the letters "banana". But we don't know yet whether they point to the same string object.

There are two possible ways the Python interpreter could arrange its memory:



In one case, a and b refer to two different objects that have the same value. In the second case, they refer to the same object. We can test whether two names refer to the same object using the is operator:

```
>>> a is b
True
```

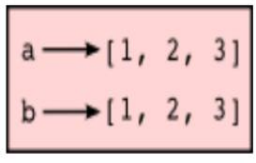
This tells us that both a and b refer to the same object, and that it is the second of the two state snapshots that accurately describes the relationship.

Since strings are immutable, Python optimizes resources by making two names that refer to the same string value refer to the same object.

This is not the case with lists:

```
>>> a = [1, 2, 3]
>>> b = [1, 2, 3]
>>> a == b
True
>>> a is b
False
```

The state snapshot here looks like this:



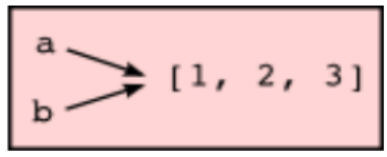
a and b have the same value but do not refer to the same object.

2.3.10 Aliasing

Since variables refer to objects, if we assign one variable to another, both variables refer to the same object:

```
>>> a = [1, 2, 3]
>>> b = a
>>> a is b
True
```

In this case, the state snapshot looks like this:



Because the same list has two different names, a and b, we say that it is aliased. Changes made with one alias affect the other:

```
>>> b[0] = 5
>>> a
[5, 2, 3]
```

Aliasing in Python

1. Aliasing occurs when two or more variables refer to the same object in memory.
   ➤ Example:
2. a = [1, 2, 3]
3. b = a # b is an alias of a

Now, if we change b, the list a also changes because both point to the same list.

4. This behavior can sometimes be useful, as it allows multiple names to refer to the same data.
5. However, it can also be unexpected or undesirable, especially when you modify one variable and accidentally change another.
6. It is safer to avoid aliasing when working with mutable objects such as:
   - Lists
   - Dictionaries
   - Sets
   - (Later, custom class objects)
7. For immutable objects (like strings and tuples), aliasing is not a problem, because:
   - Their values cannot be changed once created.
   - Therefore, no unexpected side effects occur when using an alias.

8. Example (immutable aliasing):

   s1 = "hello"

   s2 = s1

   s2 = s2.upper()

9. print(s1) # Output: hello
10. print(s2) # Output: HELLO

Here, s1 remains unchanged because strings are immutable.

11. Python sometimes automatically aliases immutable data (like strings or small numbers) internally to save memory, since there's no risk of accidental modification

Cloning Lists

1. Sometimes, we may want to modify a list but also keep a copy of the original.
   ➤ In such cases, we need to create a separate copy of the list — not just another reference to the same list.
2. This process is called cloning, to avoid confusion with the term *copy* (which can also mean reference).
3. The simplest way to clone a list in Python is by using the slice operator ([:]).
4. Example:

   a = [1, 2, 3]

   b = a[:] # Cloning the list

   print(b)

Output:

[1, 2, 3]

5. When you take a slice of a list, Python creates a new list object that contains the same elements as the original.
6. Therefore, a and b are two different lists even though they contain the same data.
7. Relationship after cloning:

   a → [1, 2, 3]

   b → [1, 2, 3]

(Both lists look the same but are stored separately in memory.)

8. Now, you can change one list without affecting the other.
9. Example:
10. b[0] = 5

11. print(a)
12. print(b)

Output:

[1, 2, 3]

[5, 2, 3]

13. As seen, modifying b does not change a. This confirms that b is a clone a true copy of the list.

2.3.12 Lists and for Loops

1. The for loop in Python is commonly used to traverse (go through) all the elements of a list. It allows us to perform actions on each element one by one.
2. General syntax:
3. for <variable> in <list>:
4. <body>
5. Example:
6. friends = ["Joe", "Zoe", "Brad", "Angelina", "Zuki", "Thandi", "Paris"]
7. for friend in friends:
8. print(friend)

Output:

Joe

Zoe

Brad

Angelina

Zuki

Thandi

Paris

This reads like English:
"For every friend in the list of friends, print the name of the friend."

Using Lists and Other Sequences in for Loops

4. You can use any list or sequence expression inside a for loop.

Example 1: Printing multiples of 3 between 0 and 19

for number in range(20):

```
    if number % 3 == 0:
        print(number)
```

Output:

0

3

6

9

12

15

18

Example 2: Using a list of fruits

```
for fruit in ["banana", "apple", "quince"]:
    print("I like to eat " + fruit + "s!")
```

Output:

I like to eat bananas!

I like to eat apples!

I like to eat quinces!

Modifying List Elements Using a for Loop

5. Since lists are mutable, we can change their elements during traversal.

Example:

```
xs = [1, 2, 3, 4, 5]
for i in range(len(xs)):
    xs[i] = xs[i] ** 2
print(xs)
```

Output:

[1, 4, 9, 16, 25]

Here, range(len(xs)) generates a sequence of indices from 0 to len(xs)-1.

6. In this loop, we need both:
    - the value of each element (to square it), and

- its index (to store the new value back in the list).

Using enumerate() for Easier Indexing

7. Python provides a built-in function enumerate() that returns both index and value during traversal.

Example:

```
xs = [1, 2, 3, 4, 5]
for (i, val) in enumerate(xs):
    xs[i] = val ** 2
print(xs)
```

Output:

[1, 4, 9, 16, 25]

8. The enumerate() function produces pairs of (index, value) for each element in the list.
9. Example (to see how enumerate works):
10. for (i, v) in enumerate(["banana", "apple", "pear", "lemon"]):
11. print(i, v)

Output:

0 banana

1 apple

2 pear

3 lemon

2.3.13 List Parameters

1. When a list is passed as an argument to a function, Python passes a reference, not a copy of the list.
   ➤ This means the function receives an alias to the original list — both refer to the same object in memory.
2. So, if the function modifies the list, the changes are visible outside the function as well.

Example:

```
def double_stuff(stuff_list):
    """Overwrite each element in a_list with double its value."""
    for (index, stuff) in enumerate(stuff_list):
```

```
    stuff_list[index] = 2 * stuff
things = [2, 5, 9]
double_stuff(things)
print(things)
```

Output:

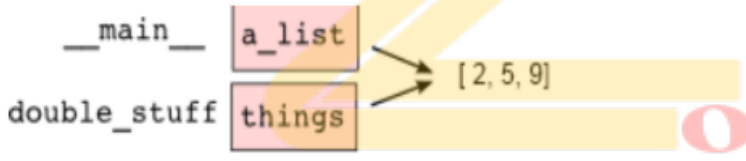
[4, 10, 18]

Explanation:

3. The variable things and the parameter stuff_list both refer to the same list object. Therefore, any modification done to stuff_list inside the function directly affects things.
4. Before modification:
5. things → [2, 5, 9]
6. stuff_list → [2, 5, 9]
7. (both point to the same list)
8. After modification:
9. things → [4, 10, 18]
10. stuff_list → [4, 10, 18]

The change made inside the function reflects in the original list.

11. Since the list object is shared, if a function modifies the list parameter, the caller sees the change.



2.3.14 List Methods

The dot operator (.) is used to access built-in methods of a list.
These methods perform specific operations on the list itself (like adding, removing, sorting, etc.).

1. append()

- Usage: mylist.append(item)
- Function: Adds an item to the end of the list.
- Example:
- mylist = []
- mylist.append(5)

- mylist.append(27)
- mylist.append(3)
- mylist.append(12)
- print(mylist)

Output: [5, 27, 3, 12]

2. insert()

- Usage: mylist.insert(index, item)
- Function: Inserts an item at the specified index; shifts the remaining items to the right.
- Example:
- mylist.insert(1, 12)
- print(mylist)

Output: [5, 12, 27, 3, 12]

3. count()

- Usage: mylist.count(item)
- Function: Returns how many times an item appears in the list.
- Example:
- mylist.count(12)

Output: 2

4. extend()

- Usage: mylist.extend([list])
- Function: Adds all elements of another list to the end of the current list.
- Example:
- mylist.extend([5, 9, 5, 11])
- print(mylist)

Output: [5, 12, 27, 3, 12, 5, 9, 5, 11]

5. index()

- Usage: mylist.index(item)
- Function: Returns the index of the first occurrence of the item.
- Example:
- mylist.index(9)

Output: 6

6. reverse()

- Usage: mylist.reverse()
- Function: Reverses the elements of the list in place.
- Example:
- mylist.reverse()
- print(mylist)

Output: [11, 5, 9, 5, 12, 3, 27, 12, 5]

7. sort()

- Usage: mylist.sort()
- Function: Sorts the list in ascending order (default).
- Example:
- mylist.sort()
- print(mylist)

Output: [3, 5, 5, 5, 9, 11, 12, 12, 27]

8. remove()

- Usage: mylist.remove(item)
- Function: Removes the first occurrence of the item from the list.
- Example:
- mylist.remove(12)
- print(mylist)

Output: [3, 5, 5, 5, 9, 11, 12, 27]

2.3.15 Pure functions and modifiers

1. Difference Between Pure Functions and Modifiers

- Pure Function:
  - Does not modify its arguments (no side effects).
  - Works only with the data passed to it.
  - Returns a new value or new list instead of changing the original.
- Modifier Function:

- Changes (modifies) the list passed as an argument.
- The changes made are called side effects.
- Example: functions that directly modify the list (like using append, remove, etc.).

2.Pure Function Example – double_stuff()

```
def double_stuff(a_list):
    """Return a new list which contains doubles of the elements in a_list."""
    new_list = []
    for value in a_list:
        new_elem = 2 * value
        new_list.append(new_elem)
    return new_list
```

3. Explanation of the Code

- A new empty list new_list is created.
- Each element in a_list is multiplied by 2.
- The doubled value is added to new_list.
- Finally, the new list is returned, leaving the original list unchanged.

Example Output

```
things = [2, 5, 9]
more_things = double_stuff(things)
print(things)       # Original list
print(more_things)  # New list returned
```

Output:

```
[2, 5, 9]
[4, 10, 18]
```

4. Important Observation

Even if you assign the result back to the same variable, it is still safe because the function returns a new list, not modifying the old one.

```
things = [2, 5, 9]
things = double_stuff(things)
print(things)
```

Output:

[4, 10, 18]

2.3.16 Functions That Produce Lists

1. Introduction

- Some functions are designed to create and return new lists instead of modifying existing ones.
- Such functions generate a new list as their output (return value).
- These are usually pure functions they do not cause side effects.

2.General Pattern to Create and Return a List

Whenever you want to write a function that produces a list, follow this pattern:

1. Initialize an empty list → result = []
2. Loop through a sequence (like range() or a list).
3. Create or calculate a new element inside the loop.
4. Append that new element to result.
5. Return the result list at the end.

Initialize → Loop → Create → Append → Return

3. Example: Function to Return Prime Numbers Less Than n

```
def primes_lessthan(n):
    """Return a list of all prime numbers less than n."""
    result = []             # Step 1: Start with an empty list
    for i in range(2, n):    # Step 2: Loop through numbers 2 to n-1
        if is_prime(i):      # Step 3: Check if number is prime
            result.append(i) # Step 4: Add it to the result list
```

```
    return result          # Step 5: Return the list
```

4. Example Execution

Suppose is_prime() is already defined.

print(primes_lessthan(10))

Output:

[2, 3, 5, 7]

5. Explanation

- The function checks each number from 2 to n-1.
- If it's prime, that number is added to the result list.
- After the loop finishes, the final list of primes is returned

Step	Action	Code Example
1	Initialize empty list	result = []
2	Loop through range	for i in range(2, n):
3	Apply condition or operation	if is_prime(i):
4	Append to list	result.append(i)
5	Return final list	return result

2.3.17 Strings and Lists

1. Relationship Between Strings and Lists

- Strings and lists are closely related in Python.
- You can convert a string into a list or combine a list into a string using built-in methods.

2. split() Method – Converting String → List

- The split() method breaks a string into a list of words (substrings).
- By default, it uses whitespace (spaces, tabs, or newlines) as the separator.

Example:

```
song = "The rain in Spain..."
words = song.split()
```

```
print(words)
```

Output:

```
['The', 'rain', 'in', 'Spain...']
```

3. Using a Delimiter in split()

- You can specify a custom delimiter (a string that marks where to split).

Example:

```
song = "The rain in Spain..."
print(song.split("ai"))
```

Output:

```
['The r', 'n in Sp', 'n...']
```

Note:
The delimiter (“ai”) does not appear in the result.

4. join() Method – Converting List → String

- The join() method does the reverse of split().
- It joins a list of strings into a single string, inserting a chosen separator (glue) between them.

Example:

```
words = ['The', 'rain', 'in', 'Spain...']
glue = ";"
phrase = glue.join(words)
print(phrase)
```

Output:

```
The;rain;in;Spain...
```

The original list words is not modified.

5. Using Different Types of Glue

Example 1: Using multi-character glue

```
" --- ".join(words)
```

Output:

'The --- rain --- in --- Spain...'

Example 2: Using no glue (empty string)

"".join(words)

Output:

'TherainінSpain...

2.3.18 Lists and Range

1. The list() Function

- Python has a built-in function called list() that tries to convert any given value into a list.

Example:

letters = list("Crunchy Frog")

print(letters)

Output:

['C', 'r', 'u', 'n', 'c', 'h', 'y', ' ', 'F', 'r', 'o', 'g']

Here, the string "Crunchy Frog" is converted into a list of individual characters.

2. Converting Back to String

- You can use the join() method to convert the list back into a string.

Example:

"".join(letters)

Output:

'Crunchy Frog'

3.Understanding range()

- The range() function represents a sequence of numbers, but it doesn't generate all numbers immediately.
- It creates a lazy object (a *promise* to produce numbers when needed).
- This is known as lazy evaluation — it saves memory and speeds up processing.

Example Function:

def f(n):

```
    """Find the first number between 101 and n that is divisible by 21"""
    for i in range(101, n):
        if i % 21 == 0:
            return i
```

Testing it:

```
print(f(110))          # Output: 105
print(f(1000000000))   # Output: 105
```

Even though range(101, 1000000000) covers a huge range,
Python doesn't create a list of all those numbers — it generates them one by one only when needed.

4. Why Lazy Evaluation Matters

- Without lazy evaluation: Python would try to build a giant list in memory → 💥 crash!
- With lazy evaluation: Python only generates numbers on demand, so it runs efficiently.

5. Forcing range() to Produce a List

- You can convert a lazy range into an actual list using list(range(...)).

Example:

```
print(range(10))        # Lazy promise
print(list(range(10)))  # Converts to actual list
```

Output:

```
range(0, 10)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

6. Fun Fact: YMMV

- YMMV stands for "Your Mileage May Vary" — meaning *your results may differ*.
- Before Python 3, range() was not lazy, so behavior could vary in older versions

2.3.19 Looping and Lists

1. **Purpose of Loops:**
   - Loops allow computers to **repeat computations quickly and accurately**.
   - They are a **core part of almost every program**, especially when dealing with large data or repetitive tasks.
2. **Unnecessary Lists (Tip):**
   - Don't create lists unless you **actually need to store** the data for later use.
   - Lists take **extra memory**, so if data is only needed temporarily (like for summing numbers), avoid storing it.
3. **Example Programs:**
   - Two functions are shown — both generate **10 million random numbers** and **return their sum**.

Program 1: Using a List

```
def sum1():
    xs = []
    for i in range(10000000):
        num = joe.randrange(1000)
        xs.append(num)  # storing all numbers in a list
    tot = sum(xs)
    return tot
```

- **Steps:**
  - Generates random numbers.
  - **Stores all numbers** in a list (xs).
  - Then calculates the sum using sum(xs).

Program 2: Without a List

```
def sum2():
    tot = 0
    for i in range(10000000):
        num = joe.randrange(1000)
```

```
        tot += num   # directly adds to total
    return tot
```

- **Steps:**
  - Generates random numbers.
  - **Directly adds** each number to the total sum.
  - **Does not store** numbers in memory.

4. **Why Prefer the Second Version (sum2)**
   - **Memory Efficiency:**
     - sum1() stores 10 million numbers → uses **huge memory**.
     - sum2() doesn't store → uses **very little memory**.
   - **Faster Execution:**
     - Less data storage = **faster program**.
   - **No Memory Errors:**
     - Large lists can cause **fatal memory errors**, especially when memory is limited.

5. **Practical Example (Files Analogy):**
   - When reading files:
     - **Whole file at once** → loads everything into memory (can be risky for large files).
     - **Line-by-line reading** → reads only one line at a time (safe and memory-efficient).
   - The **line-by-line method** is preferred for **large files**.

6. **Key Takeaway:**
   - Use lists **only if you need to reuse data** later.
   - If data is used **immediately**, process it directly inside the loop.
   - Always think about **memory usage and efficiency** when working with loops.

2.3.20 Nested Lists

1. Definition:
   - A nested list is a list that contains another list as one of its elements.

- It allows storing multiple levels of data inside one list.

2. Example:
3. nested = ["hello", 2.0, 5, [10, 20]]
   - This list has 4 elements:
     - "hello" → string
     - 2.0 → float
     - 5 → integer
     - [10, 20] → another list (this is the nested list)
4. Accessing the Nested List:
5. print(nested[3])
   - Output: [10, 20]
   - The element at index 3 is itself a list.

6. Accessing Elements Inside the Nested List (Two Steps):

   ```
   elem = nested[3]   # get the nested list
   print(elem[0])     # access its first element
   ```

   - Output: 10

7. Accessing Elements in One Step (Combined):

   ```
   print(nested[3][1])
   ```

   - Output: 20
   - Explanation:
     - nested[3] → gets [10, 20]
     - [1] → gets the second element (index 1) → 20

8. How It Works:
   - The bracket operators ([]) are evaluated from left to right.
   - So, nested[3][1] means:
     - Access the 3rd element of nested.
     - Then access the 1st element inside that nested list.

Key Takeaway:

- Nested lists help store complex data structures.
- You can access elements step-by-step or in a single combined expression.
- Always remember: first access the outer list, then the inner one.

2.3.21 Matrices

1. **Definition:**
   - A **matrix** is a **rectangular arrangement of numbers** in rows and columns.
   - In Python, we can represent a matrix using **nested lists** — each **inner list represents one row** of the matrix.
2. **Example Matrix:**



This matrix can be written in Python as:

mx = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

3. **Structure:**
   - The **outer list** → represents the matrix as a whole.
   - Each **inner list** → represents one **row**.

So:

- mx[0] → [1, 2, 3] (1st row)
- mx[1] → [4, 5, 6] (2nd row)
- mx[2] → [7, 8, 9] (3rd row)

4. **Accessing a Row:**

   print(mx[1])

Output:

[4, 5, 6]

- mx[1] selects the **2nd row** (since indexing starts from 0).

5. **Accessing a Single Element (Double Index):**

print(mx[1][2])

Output:

6

- First index 1 → selects **2nd row** [4, 5, 6]
- Second index 2 → selects **3rd element** in that row → 6

6. **How Indexing Works:**
   - mx[row][column]
   - Example: mx[2][0] → 7 (3rd row, 1st column)

7. **Alternate Representation:**
   - Instead of rows, you could represent a matrix as a **list of columns**:
   - mx_col = [[1, 4, 7], [2, 5, 8], [3, 6, 9]]
   - Here, each inner list is a **column** instead of a row.
   - (This approach is less common.)

8. **Future Note:**
   - Later, we can represent matrices more efficiently using:
     - **Dictionaries** (for sparse matrices)
     - **NumPy arrays** (for numerical operations)

Key Takeaway:

- A **matrix = list of lists** in Python.
- **First index = row**, **second index = column**.
- Example:

mx[1][2] → element from 2nd row, 3rd column = 6

2.3.22 Glossary

aliases Multiple variables that contain references to the same object.

clone To create a new object that has the same value as an existing object. Copying a reference to an object creates an alias but doesn't clone the object.

delimiter A character or string used to indicate where a string should be split.

element One of the values in a list (or other sequence). The bracket operator selects elements of a list. Also called item.

immutable data value A data value which cannot be modified. Assignments to elements or slices (sub-parts) of immutable values cause a runtime error.

index An integer value that indicates the position of an item in a list. Indexes start from 0. item See element.

list A collection of values, each in a fixed position within the list. Like other types str, int, float, etc. there is also a list type-converter function that tries to turn whatever argument you give it into a list.

list traversal The sequential accessing of each element in a list.

modifier A function which changes its arguments inside the function body. Only mutable types can be changed by modifiers.

mutable data value A data value which can be modified. The types of all mutable values are compound types. Lists and dictionaries are mutable; strings and tuples are not.

nested list A list that is an element of another list.

object A thing to which a variable can refer.

pattern A sequence of statements, or a style of coding something that has general applicability in a number of different situations. Part of becoming a mature Computer Scientist is to learn and establish the patterns and algorithms hat form your toolkit. Patterns often correspond to your "mental chunking".

promise An object that promises to do some work or deliver some values if they're eventually needed, but it lazily puts off doing the work immediately. Calling range produces a promise.

pure function A function which has no side effects. Pure functions only make changes to the calling program through their return values.

sequence Any of the data types that consist of an ordered collection of elements, with each element identified by an index.

side effect A change in the state of a program made by calling a function. Side effects can only be produced by modifiers.

step size The interval between successive elements of a linear sequence. The third (and optional argument) to the range function is called the step size. If not specified, it defaults to 1.

Thank You

We're glad to be part of your engineering journey.
Keep exploring, keep innovating, and keep growing.

AZ Documents

Your Engineering Study Partner

www.azdocuments.in

Quality notes, simplified explanations, and student-focused resources for every semester.

Connect With Us:

www.azdocuments.in [Click Here]

@azdocuments [Click Here]

Join Our Student Community: [Click Here]

© 2025 AZ Documents. All rights reserved.

This material is intended for educational purposes only.
Redistribution without permission is prohibited.